

1) Source of Truth en BD: tabla debacu_eval_subscriptions

Si ahora mismo tienes subscriptions genérica, yo haría **tabla propia** (o al menos una vista/alias) para no mezclar con otras apps. Si prefieres no migrar, replica los campos clave en la actual. Pero el diseño “definitivo” es este:

```
-- 1) Suscripciones (verdad operativa)
create table if not exists public.debacu_eval_subscriptions (
  id uuid primary key default gen_random_uuid(),

  customer_id uuid not null,
  app_id text not null default 'debacu_eval',

  status text not null check (status in (
    'TRIAL_ACTIVE',
    'ACTIVE',
    'PENDING_PAYMENT',
    'PAST_DUE',
    'CANCELED',
    'SUSPENDED'
  )),

  plan_code text not null check (plan_code in
('FREE', 'BASIC', 'MEDIUM', 'PREMIUM')),

  billing_frequency text null check (billing_frequency in ('MONTH', 'YEAR')),
  required_plan_code text null check (required_plan_code in
('FREE', 'BASIC', 'MEDIUM', 'PREMIUM')),
  required_billing_frequency text null check (required_billing_frequency in
('MONTH', 'YEAR')),

  start_date date null,
  end_date date null,
  next_billing_date date null,
  trial_ends_at timestamptz null,
  grace_ends_at timestamptz null,
  suspended_at timestamptz null,

  provider text not null default 'STRIPE' check (provider in ('STRIPE')),
  stripe_customer_id text null,
  stripe_subscription_id text null,
  stripe_schedule_id text null,
  stripe_price_id text null,
  stripe_checkout_session_id text null,

  replaces_subscription_id uuid null references
public.debacu_eval_subscriptions(id),

  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index if not exists debacu_eval_subscriptions_customer_idx
on public.debacu_eval_subscriptions(customer_id);

create index if not exists debacu_eval_subscriptions_status_idx
on public.debacu_eval_subscriptions(status);

-- Regla: 1 suscripción "vigente" por customer/app (vigente = no CANCELED)
-- Nota: Postgres partial unique index
```

```
create unique index if not exists debacu_eval_subscriptions_one_current
on public.debacu_eval_subscriptions(customer_id, app_id)
where status in
('TRIAL_ACTIVE', 'ACTIVE', 'PENDING_PAYMENT', 'PAST_DUE', 'SUSPENDED');
```

Por qué así:

- Tu app decide permisos con esto, no preguntando a Stripe.
 - `required_*` permite downgrade/upgrade “programado”.
 - El índice parcial evita el caos de múltiples activas.
-

2) Eventos (auditoría técnica): `debacu_eval_subscription_events`

Esto te da trazabilidad e idempotencia del webhook.

```
create table if not exists public.debacu_eval_subscription_events (
  id uuid primary key default gen_random_uuid(),
  customer_id uuid null,
  subscription_id uuid null references public.debacu_eval_subscriptions(id),

  provider text not null default 'STRIPE',
  event_type text not null, -- e.g. checkout.session.completed, invoice.paid...
  provider_event_id text not null, -- stripe event id
  provider_object_id text null, -- sub id, checkout id, invoice id...

  payload jsonb not null default '{}':::jsonb,

  created_at timestamptz not null default now()
);

-- idempotencia: no procesar dos veces el mismo evento
create unique index if not exists
debacu_eval_subscription_events_provider_event_unique
on public.debacu_eval_subscription_events(provider, provider_event_id);
```

3) Control de seats: base de datos (users del customer)

Necesitas una tabla de “usuarios del customer” que puedas contar. Si ya la tienes, perfecto. Si no, mínimo:

```
create table if not exists public.debacu_eval_customer_users (
  id uuid primary key default gen_random_uuid(),
  customer_id uuid not null,
  user_id uuid not null, -- auth.users.id o tu tabla users
  role text not null default 'USER' check (role in ('OWNER', 'ADMIN', 'USER')),
  status text not null default 'ACTIVE' check (status in ('ACTIVE', 'DISABLED')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

  unique(customer_id, user_id)
);

create index if not exists debacu_eval_customer_users_customer_idx
```

```
on public.debacu_eval_customer_users(customer_id);

create index if not exists debacu_eval_customer_users_status_idx
on public.debacu_eval_customer_users(status);
```

4) Lógica de seats y permisos: SIEMPRE en Edge Functions

4.1 Mapa plan → seats (centralizado)

- FREE: 1
- BASIC: 1
- MEDIUM: 2
- PREMIUM: 4

4.2 Reglas duras

- Si `status` $\notin$ {`TRIAL_ACTIVE`, `ACTIVE`} $\Rightarrow$ no permites acciones “core” (crear evaluación, exportar, etc.)
 - Para invitar/activar usuario $\Rightarrow$ contar `ACTIVE` y comparar con max.
-

5) Helper estándar (Deno) para TODAS las Edge Functions

Crea un fichero común en `supabase/functions/_shared/plan.ts`:

```
// supabase/functions/_shared/plan.ts
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

export type PlanCode = "FREE" | "BASIC" | "MEDIUM" | "PREMIUM";
export type SubStatus =
  | "TRIAL_ACTIVE"
  | "ACTIVE"
  | "PENDING_PAYMENT"
  | "PAST_DUE"
  | "CANCELED"
  | "SUSPENDED";

export function planMaxUsers(plan: PlanCode): number {
  switch (plan) {
    case "FREE":
    case "BASIC":
      return 1;
    case "MEDIUM":
      return 2;
    case "PREMIUM":
      return 4;
  }
}

export function isAppEnabled(status: SubStatus): boolean {
  return status === "ACTIVE" || status === "TRIAL_ACTIVE";
}
```

```

export function isSeatBlockingStatus(status: SubStatus): boolean {
  // para altas de usuario, normalmente solo dejamos si enabled
  return !isAppEnabled(status);
}

export function supabaseAdmin(req: Request) {
  // Service role SOLO en Edge Function (nunca en frontend)
  const url = Deno.env.get("SUPABASE_URL")!;
  const key = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;
  return createClient(url, key, {
    global: { headers: { Authorization: req.headers.get("Authorization") ?? "" }
  },
  });
}

export function supabaseUser(req: Request) {
  const url = Deno.env.get("SUPABASE_URL")!;
  const anon = Deno.env.get("SUPABASE_ANON_KEY")!;
  return createClient(url, anon, {
    global: { headers: { Authorization: req.headers.get("Authorization") ?? "" }
  },
  });
}

export async function getAuthUserOrThrow(sbUser: ReturnType<typeof
supabaseUser>) {
  const { data, error } = await sbUser.auth.getUser();
  if (error || !data?.user) throw new Error("UNAUTHENTICATED");
  return data.user;
}

// NOTA: aquí asumo que tienes forma de obtener customer_id del usuario.
// Si hoy lo tienes vía tabla perfil, ajustas el query.
export async function getCustomerIdForUserOrThrow(
  sbAdmin: ReturnType<typeof supabaseAdmin>,
  userId: string
): Promise<string> {
  // EJEMPLO: tabla debacu_eval_profiles(user_id, customer_id)
  const { data, error } = await sbAdmin
    .from("debacu_eval_profiles")
    .select("customer_id")
    .eq("user_id", userId)
    .maybeSingle();

  if (error) throw error;
  if (!data?.customer_id) throw new Error("NO_CUSTOMER");
  return data.customer_id as string;
}

export type ActiveSub = {
  id: string;
  customer_id: string;
  plan_code: PlanCode;
  status: SubStatus;
  next_billing_date: string | null;
  trial_ends_at: string | null;
  grace_ends_at: string | null;
};

export async function getCurrentSubscriptionOrThrow(
  sbAdmin: ReturnType<typeof supabaseAdmin>,
  customerId: string
): Promise<ActiveSub> {

```

```

const { data, error } = await sbAdmin
  .from("debacu_eval_subscriptions")
  .select("id, customer_id, plan_code, status, next_billing_date,
trial_ends_at, grace_ends_at")
  .eq("customer_id", customerId)
  .eq("app_id", "debacu_eval")
  .in("status", ["TRIAL_ACTIVE", "ACTIVE", "PENDING_PAYMENT", "PAST_DUE",
"SUSPENDED"])
  .order("created_at", { ascending: false })
  .limit(1)
  .maybeSingle();

if (error) throw error;
if (!data) throw new Error("NO_SUBSCRIPTION");
return data as unknown as ActiveSub;
}

export async function assertAppEnabledOrThrow(sub: ActiveSub) {
  if (!isAppEnabled(sub.status)) {
    throw new Error(`PLAN_NOT_ACTIVE:${sub.status}`);
  }
}

export async function assertSeatAvailableOrThrow(
  sbAdmin: ReturnType<typeof supabaseAdmin>,
  customerId: string,
  plan: PlanCode
) {
  const max = planMaxUsers(plan);

  const { count, error } = await sbAdmin
    .from("debacu_eval_customer_users")
    .select("id", { count: "exact", head: true })
    .eq("customer_id", customerId)
    .eq("status", "ACTIVE");

  if (error) throw error;

  const used = count ?? 0;
  if (used >= max) throw new Error(`SEATS_EXCEEDED:${used}/${max}`);
}

```

Esto te da un “middleware” real: autenticación, customer_id, suscripción, seats.

6) Ejemplo de Edge Function usando el helper

debacu_eval_users_invite_or_create (bloquea si no hay seats)

```

import { corsHeaders, json } from "../_shared/http.ts"; // asumiendo que ya lo
tienes
import {
  supabaseAdmin,
  supabaseUser,
  getAuthUserOrThrow,
  getCustomerIdForUserOrThrow,
  getCurrentSubscriptionOrThrow,
  assertSeatAvailableOrThrow,
  assertAppEnabledOrThrow,
} from "../_shared/plan.ts";

```

```

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return new Response(null, { headers:
corsHeaders(req) });

  try {
    const sbUser = supabaseUser(req);
    const sbAdmin = supabaseAdmin(req);

    const user = await getAuthUserOrThrow(sbUser);
    const customerId = await getCustomerIdForUserOrThrow(sbAdmin, user.id);

    const sub = await getCurrentSubscriptionOrThrow(sbAdmin, customerId);
    await assertAppEnabledOrThrow(sub);
    await assertSeatAvailableOrThrow(sbAdmin, customerId, sub.plan_code);

    const body = await req.json(); // email, role...
    // aquí creas invitación / usuario y registras en debacu_eval_customer_users

    return json(req, 200, { ok: true });
  } catch (e) {
    const msg = e instanceof Error ? e.message : "UNKNOWN";
    return json(req, 400, { ok: false, error: msg });
  }
});

```

7) Punto crítico: tu problema actual (subscription_events con customerId/appId)

En tu continuidad técnica ya dijiste que tienes:

- **pendiente corregir** subscription_events por columnas customerId/appId vs snake_case/minúsculas

Mi recomendación directa:

- En BD: **snake_case** (customer_id, app_id)
- En frontend: camelCase
- En Edge: tú decides, pero lo normal es camelCase en payload y haces mapping al insertar

Si ahora mismo tienes columnas con comillas "customerId", eso te va a dar dolor constante. Solución limpia: migración para normalizar.

8) Lo siguiente que hacemos ya (sin preguntas, por orden)

1. **Migración:** crear/ajustar debacu_eval_subscriptions + debacu_eval_subscription_events + debacu_eval_customer_users (o adaptar las tuyas).
2. **Helper _shared/plan.ts** metido en tu repo.
3. **Actualizar 2 Edge Functions clave:**

- `debacu_eval_subscription_get_active` (para UI)
- `debacu_eval_users_invite_or_create` (para seats)

4. Luego: webhook `stripe_webhook_subscriptions` idempotente (usando `provider_event_id` unique).